Lab 1 – Characterisation of the Data-Link Protocol Efficiency

Luís Kong (up202409115) · André Pereira (up202405143)
Class 3LEIC15 · Redes de Computadores (RC), L.EIC – FEUP, 2026/2027

Summary

We measured the efficiency S of our Stop-and-Wait ARQ data-link protocol in 108 file transfers over a virtual serial cable, varying the baud rate (C), the propagation delay (T_{prop}), the frame error rate (FER) and the frame size (L) one at a time. The error-free baseline at 9600 bit/s reaches S = 0.779. The theoretical formula S = (1 − FER)/(1 + 2a) predicts the trends but overestimates S by a factor of 1.28; a corrected model that counts start and stop bits, frame overhead and replies matches every error-free point above 2400 bit/s to within 0.001. Outside both models, the fixed 4 s timer halves S at 2400 bit/s and below, and BCC2 missed the errors in 4 of 34 distinct noisy transfers.

Introduction

Our protocol provides reliable file transfer over an RS-232 link, with an application layer and a data-link layer that stay independent through the functions `llOpenTx`, `llOpenRx`, `llSend`, `llReceive`, `llCloseTx` and `llCloseRx`. Every frame is delimited by FLAG (0x7E) and carries an address, a control field and a header check BCC1. Information frames also carry a packet and a data check BCC2, the XOR of the packet bytes. Byte stuffing escapes 0x7E and 0x7D inside the frame.

The receiver answers each correct frame with RR and each frame with a bad BCC2 with REJ. The transmitter resends a frame at once after a REJ, or after a 4 s timeout, and gives up on a frame after 4 timeouts (3 retransmissions); REJs do not count towards this limit. Headers with a bad BCC1 are ignored, so an error there or in a reply costs a timeout.

For a frame of L bytes, the theoretical efficiency of Stop-and-Wait is

$$S = \frac{1 - \mathrm{FER}}{1 + 2a}, \quad a = \frac{T_{\mathrm{prop}}}{T_f}, \quad T_f = \frac{8L}{C}, \quad \mathrm{FER} \approx 1 - (1 - \mathrm{BER})^{8L} \qquad (1)$$

Methodology

Setup. All transfers ran on one macOS computer over the course's `cable` program, which sets C, T_{prop} and the bit error rate (BER), and applies bit errors in both directions. Each transfer sent `penguin.gif` (10 968 bytes) using our code without changes. For the L tests, we built copies of the source in a separate folder with a different data chunk size. L is the packet carried by an I frame: a 3-byte header plus L − 3 file bytes.

Measurements. The transmitter times each transfer from the UA that opens the link to the start of the disconnection, so the time T covers the START packet, the data and the END packet. We compute R = 8 × (file bytes) / T and S = R / C, and take the measured FER as retransmissions / (frames accepted + retransmissions). Each point is the mean of the repetitions in Table 1, and we compared the output with the original file after every transfer. The baseline is C = 9600 bit/s, T_{prop} = 0, BER = 0 and L = 1000 bytes.

Table 1: Experiments. Parameters not listed keep the baseline value.

Parameter	Values	Runs
C (bit/s)	1200, 2400, 4800, 9600, 19 200, 38 400, 57 600, 115 200	3 × 8
T_{prop} (ms)	0, 10, 50, 100, 200, 500	3 × 6
FER (via BER)	targets 0, 0.1, 0.2, 0.3, 0.4, 0.5; BER from Equation 1	5 × 6
L (bytes)	125, 250, 500, 1000, at T_{prop} = 0, at T_{prop} = 100 ms and at BER = $2.79 \cdot 10^{-5}$	3 × 12

Corrected model. Equation 1 treats every bit on the line as useful. On our link each byte takes 10 bit times (8-N-1), and the transfer includes headers, stuffing, control packets and one 5-byte reply per frame. With n_i bytes on the line for frame i, counted from the actual file, the error-free time and the efficiency are

$$T_0 = \sum_i \left[\frac{10(n_i + 5)}{C} + 2T_{\mathrm{prop}} \right], \quad S_c = \frac{8F}{C\,T_0}(1 - \mathrm{FER}) \qquad (2)$$

where F is the file size in bytes. With noise, each REJ adds one frame cycle $T_c = 10(n + 5)/C$ and each timeout adds 4 s to T_0.

Results

Figure 1 shows the measured efficiency with both models, and Table 2 gives the differences. All 108 transfers reached the end of the file, and 103 delivered an identical copy.

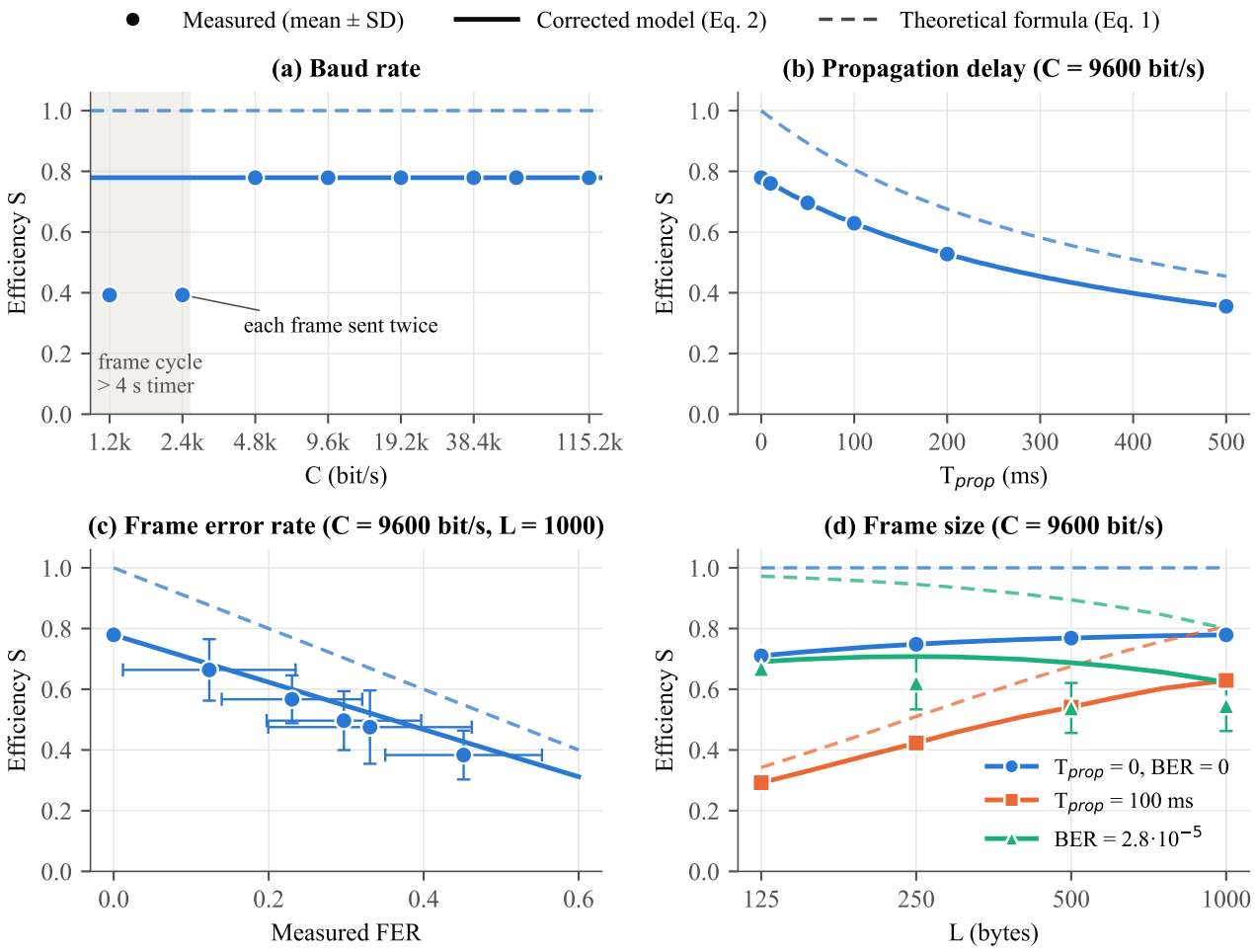


Figure 1: Measured efficiency S against (a) C, (b) T_{prop}, (c) measured FER and (d) L, with Equation 1 (theoretical) and Equation 2 (corrected). Colour marks the test condition; solid lines show Equation 2 and dashed lines Equation 1. Error bars show the standard deviation of the repetitions, and the shaded band in (a) marks the rates where a frame cycle exceeds the 4 s timer. In (d), the BER curves use the FER predicted by Equation 1.

Table 2: Mean absolute difference between measured S and each model. The C row excludes 1200 and 2400 bit/s.

Experiment	Equation 1	Equation 2
C (T_{prop} = 0)	0.221	< 0.001
T_{prop}	0.177	< 0.001
L, T_{prop} = 0 / 100 ms	0.248 / 0.112	< 0.001 / < 0.001
FER (using measured FER)	0.200	0.032
L, BER = $2.79 \cdot 10^{-5}$ (using measured FER)	0.259	0.045

Baseline. The error-free baseline gives T = 11.73 s, R = 7479 bit/s and S = 0.779. The transfer puts 11 262 bytes on the line for 10 968 file bytes, so 8 × 10 968 / (10 × 11 262) = 0.779 is its upper limit, and the protocol reaches it. The 10-bit byte alone limits S to 0.8.

Baud rate. S stays at 0.779 from 4800 to 115 200 bit/s, as a ≈ 0 predicts. At 2400 and 1200 bit/s it falls to 0.393 and 0.392, because the largest frame and its reply take 4.27 s and 8.54 s, more than the 4 s timer. The transmitter resends frames that are still arriving: these runs show 12 and 13 retransmissions, the receiver discards the same number of duplicates, and T doubles to 93.1 s and 186.3 s. With this timer, 1000-byte frames need C above about 2560 bit/s.

Propagation delay. S falls from 0.779 at T_{prop} = 0 to 0.629 at 100 ms and 0.355 at 500 ms (a = 0.6), because the transmitter stays idle for each round trip. Equation 1 follows this shape, staying 1.28 times above the measurements at every delay. Equation 2 matches all six points.

Frame error rate. S falls roughly linearly with FER, from 0.779 to 0.383 at a measured FER of 0.452 (Table 3). The file needs only 14 frames, so the measured FER varies between runs, with a standard deviation of up to 0.13. Measured S lies 0.019 to 0.051 below Equation 2 because the FER counts all 14 frames, including 3 short ones that errors rarely hit, while almost every retransmission repeats a full 1.06 s frame cycle. Timeouts

were rare, 0.2 per transfer at two points, but each costs 4 s. Counting REJs and timeouts directly reproduces T in all 37 noisy runs to within 0.16 s.

Table 3: Efficiency against FER (5 runs per point; C = 9600 bit/s, T_{prop} = 0, L = 1000 bytes). REJs and timeouts are means per transfer, counting only the data transfer.

BER	Measured FER	REJs	Timeouts	T (s)	S (mean ± SD)	S_c	Corrupted files
0	0	0	0	11.73	0.779 ± 0.000	0.779	0
$1.32 \cdot 10^{-5}$	0.123	2.2	0	14.06	0.664 ± 0.101	0.683	0
$2.79 \cdot 10^{-5}$	0.230	4.4	0	16.39	0.567 ± 0.079	0.600	1
$4.46 \cdot 10^{-5}$	0.297	6.0	0.2	18.89	0.497 ± 0.097	0.548	1
$6.39 \cdot 10^{-5}$	0.331	7.4	0.2	20.38	0.475 ± 0.121	0.521	1
$8.66 \cdot 10^{-5}$	0.452	12.2	0	24.65	0.383 ± 0.081	0.427	1

Frame size. Without errors, longer frames raise S from 0.710 at L = 125 to 0.779 at L = 1000, because overhead takes a smaller share; Equation 1 gives S = 1 for every L. With T_{prop} = 100 ms the effect is stronger, from 0.292 to 0.629, because a falls as L grows. With BER = $2.79 \cdot 10^{-5}$ the order reverses: L = 125 reaches S = 0.668 ± 0.022 against 0.546 ± 0.084 at L = 1000, as the measured FER grows from 0.054 to 0.254. The best L therefore depends on the link: 1000 bytes without errors, with or without delay, and 125 bytes among those tested at this BER.

Undetected errors. The noisy FER runs delivered 4 corrupted files in 25 transfers, although S still counts their time. We replayed one and found two flipped bits in the same bit position of the same frame, which cancel in the XOR of BCC2. The cable's noise generator is unseeded, so the three L = 1000 runs in the noisy L series repeat the first three runs at the same BER, including one of these corrupted files. They add no independent evidence, which leaves 34 distinct noisy transfers.

Conclusions

The measurements confirm the behaviour that Equation 1 describes: S falls roughly linearly with FER and as 1/(1 + 2a) with delay, but the formula overestimates S by a factor of 1.28 on this link. Counting start and stop bits, frame overhead and replies in Equation 2 reduces the mean error on error-free points below 0.001. In the ranges tested, the lowest S came from T_{prop} = 500 ms (0.355). A timer that scales with the frame time would remove the loss at 2400 bit/s and below, and a CRC in place of the XOR would detect the errors that reach the file. These results describe the virtual cable, where errors are independent and the delay is exact; a real serial link may behave differently.